Now all you need is the mailer. You can build a mailer from the command line using a Rails generator:

```
$ rails generate mailer TaskMailer
      create  app/mailers/task_mailer.rb
      invoke  erb
      create    app/views/task_mailer
      invoke  rspec
      create    spec/mailers/task_mailer_spec.rb
      create    spec/mailers/previews/task_mailer_preview.rb
```

Note that RSpec creates its own mailer files (I'll come back to those in a moment). I won't cover the preview file here, as that's not strictly an automated-testing thing. (You can find details online.[5])

The mailer itself is straightforward; you need to take the task object and set some mail variables:

display/04/app/mailers/task_mailer.rb

```
class TaskMailer < ApplicationMailer
  default from: "from@example.com"

  def task_completed_email(task)
    @task = task
    mail(to: "monitor@tasks.com", subject: "A task has been completed")
  end
end
```

And you need a template. Let's keep this simple:

display/04/app/views/task_mailer/task_completed_email.text.erb

```
The task <%= @task.title %> was completed at <%= @task.completed_at.to_s %>

Thanks,

The Management
```

The tests pass.

You can also unit-test the mailer. This example tests the same functionality:

display/04/spec/mailers/task_mailer_spec.rb

```
require "rails_helper"

RSpec.describe TaskMailer, type: :mailer do

  describe "task_completed_email" do
    let(:task) { create(:task, title: "Learn how to test mailers", size: 3) }
    let(:mail) { TaskMailer.task_completed_email(task) }

    it "renders the email" do
      expect(mail.subject).to eq("A task has been completed")
```

5. http://guides.rubyonrails.org/action_mailer_basics.html#previewing-emails